[Portada del equipo]

Reemplazar esta página por la portada. El índice ya cuenta esta página como la 1.

Índice

1. Introducción **4**
1.1 Objetivo 4
1.2 Resultados esperados y estado 4
2. Funcionalidades **5**
3. Tecnologías utilizadas **6**
4. Arquitectura **7**
4.1 Vista general 7
4.2 Estructura de carpetas 7
4.3 Renderizado en el cliente (CSR) y en el servidor (SSR) 7
4.4 Flujo de una reseña 8
4.5 Estrategias de caché del service worker 8
4.6 Datos de Jalpan de Serra y sus fuentes 8
4.7 Modelo de datos 9
5. Desarrollo por fases **11**
Fase 0 · Entorno de desarrollo y repositorio 11
Fase 1 · App shell 11
Fase 2 · Manifiesto e íconos 12
Fase 3 · Service worker básico 12
Fase 4 · Renderizado CSR y SSR 13
Fase 5 · Splash y Home 13
Fase 6 · Datos locales 14
Fase 7 · Estrategias de caché y offline 14
Fase 8 · Reseñas y Background Sync 15
Fase 9 · Ciclo de vida del service worker 16
Fase 10 · Notificaciones locales y push 17
Fase 11 · Dispositivo e hidratación 18
Fase 12 · Pruebas 18
Fase 13 · Publicación 18
6. Mejoras posteriores **20**
6.1 Mapa real, recorridos y «Cómo llegar» 20
6.2 Datos reales de Jalpan 21
6.3 Instalación más robusta 21
6.4 Diseño con identidad local 21
6.5 Reseñas reales con Supabase 22
7. Pruebas **23**
7.1 Herramientas 23
7.2 Pruebas automáticas del servidor 23
7.3 Casos de prueba completos 23
7.4 Auditoría Lighthouse 25
8. Publicación **26**
8.1 Repositorio y GitHub Pages 26
8.2 Qué funciona en GitHub Pages y qué necesita el servidor 26
8.3 Cómo publicar una actualización 26
9. Capturas de pantalla **27**
10. Créditos de imágenes y datos **33**

11. Limitaciones y trabajo futuro **34**
12. Conclusiones **35**
Anexo A. Enlaces y comandos **36**
Anexo B. Historial de commits **36**

1. Introducción

RutaExpress es una Aplicación Web Progresiva (PWA) que funciona como guía del transporte de Jalpan de Serra, en la Sierra Gorda de Querétaro. Reúne las salidas de autobús desde Jalpan hacia otros destinos: empresa, horario de primera y última salida, número de salidas, duración y tarifa. También incluye comentarios de los usuarios en tiempo real, avisos de retrasos o desvíos y la parada más cercana por GPS.

En la sierra la señal de celular es intermitente, por eso el funcionamiento sin conexión no es un adorno: la aplicación abre, muestra el catálogo y guarda los comentarios aunque no haya internet, y los envía sola cuando la red regresa.

1.1 Objetivo

Construir paso a paso una Aplicación Web Progresiva completa, desde el entorno de desarrollo hasta su publicación, aplicando app shell, manifiesto, service worker, almacenamiento local, funcionamiento offline, sincronización en segundo plano y notificaciones push. Al terminar cada fase se verificó su funcionamiento y se registró un commit en el repositorio.

1.2 Resultados esperados y estado

Resultado esperado	Estado	Evidencia
La PWA se instala	Cumplido	Manifiesto e instalabilidad sin errores en Edge (CP-03, CP-26); botón «Instalar app» con instrucciones de respaldo (CP-09)
Funciona sin conexión	Cumplido	Con el servidor apagado: catálogo, fichas visitadas y no visitadas (CP-13 a CP-15)
Sincroniza reseñas al recuperar la red	Cumplido	Reseña escrita offline enviada por Background Sync al reconectar (CP-17)
Recibe notificaciones push	Cumplido	Suscripción real con VAPID y entrega por el servicio push de Windows (CP-21)
Repositorio con al menos un commit por fase	Cumplido	24 commits: uno por cada fase 0 a 13 más las mejoras (Anexo B)
Aplicación publicada en GitHub Pages	Cumplido	https://kaidav54.github.io/pwaauto/ con despliegue automático (CP-26)

> **Atención a las observaciones de la revisión**
> (1) La tabla completa de los 29 casos de prueba y la auditoría Lighthouse están en la sección 7. (2) El índice tiene números de página. (3) Cada fase incluye los fragmentos clave de su código. (4) Los datos ahora son de Jalpan de Serra con sus fuentes (sección 4.6) y todas las fotos tienen créditos (sección 10).

2. Funcionalidades

La aplicación tiene cinco secciones en el menú inferior: Inicio, Paradas, Cerca, Avisos y Ajustes.

Función	Qué hace	Tecnología principal
Catálogo (CSR)	Lista con buscador por destino, empresa o nombre y mapa sincronizado	Enrutador por hash, fetch, Leaflet
Ficha de parada (SSR)	Salidas con empresa, primera y última salida, número de salidas, duración y tarifa; fuentes de cada dato	Plantilla en el servidor + hidratación
Recorrido de cada corrida	Al tocar una corrida se dibuja su trazo por carretera hasta el destino	Leaflet + trazos OSRM precalculados
Cómo llegar	Camino a pie por calles desde tu ubicación hasta la parada, sin salir de la app	Geolocation + OSRM peatón
Cerca de mí	Parada más cercana con distancia, rumbo y minutos caminando	Geolocation API, fórmula de Haversine
Reseñas en tiempo real	Comentarios que llegan al instante a otros celulares; botón para reportar	Supabase (PostgreSQL + Realtime)
Reseñas sin conexión	El comentario se guarda en el celular y se envía solo al volver la señal	IndexedDB + Background Sync
Avisos push	Retrasos, desvíos y cambios de andén aunque la app esté cerrada	Push API, VAPID, web-push
Favoritas y última consultada	Accesos rápidos en el Inicio	IndexedDB, localStorage
Instalación y actualizaciones	Ícono en el teléfono, splash y aviso «Hay una nueva versión»	Manifiesto, ciclo de vida del SW
Tema claro/oscuro, compartir y vibrar	Preferencia guardada; hoja nativa de compartir; vibración al confirmar	localStorage, Web Share, Vibration

3. Tecnologías utilizadas

Capa	Tecnología	Uso en el proyecto
Servidor	Node.js 24 + Express 4.22	Archivos estáticos, API, ficha SSR, API de reseñas de respaldo y envío de push
Push	web-push 3.6 (VAPID)	Firmar y enviar notificaciones a los navegadores suscritos
Base de datos	Supabase (PostgreSQL)	Reseñas compartidas con Row Level Security, antispam y reportes
Tiempo real	Supabase Realtime (supabase-js 2.117.3)	Avisar a los celulares cuando llega una reseña nueva
Mapas y rutas	Leaflet 1.9.4, OpenStreetMap, OSRM, Nominatim	Mapas, ubicaciones, trazos por carretera y caminos a pie
Navegador	Service Worker, Cache Storage, IndexedDB, localStorage	Caché del shell, datos sin conexión, cola de envíos y preferencias
APIs del dispositivo	Geolocation, Notifications, Push, Background Sync, Vibration, Web Share	Funciones físicas y en segundo plano
Interfaz	HTML, CSS y JavaScript (módulos ES) sin frameworks	App shell, vistas y componentes
Diseño	Bricolage Grotesque y Atkinson Hyperlegible (OFL); íconos SVG propios	Identidad visual de la Sierra Gorda
Imágenes	sharp	Convertir fotos a WebP optimizado en varios tamaños
Pruebas	node:test, Edge + Chrome DevTools Protocol, Lighthouse 13.5	Pruebas automáticas, de extremo a extremo y auditoría
Publicación	Git, GitHub, GitHub Actions, GitHub Pages	Control de versiones y despliegue automático

4. Arquitectura

4.1 Vista general

El proyecto tiene tres partes que se comunican por HTTP:

- **Cliente PWA (public/).** Lo único que se publica en GitHub Pages: shell, vistas, service worker, manifiesto, datos, imágenes y fuentes.
- **Servidor Node (server.js).** Genera la ficha SSR, expone la API y envía las notificaciones push. Se ejecuta con npm start.
- **Servicios externos.** Supabase (reseñas), OpenStreetMap (mapas), OSRM (caminos a pie) y el servicio push del navegador (FCM o WNS).

La aplicación detecta si hay servidor y se adapta: con servidor usa la ficha SSR y el push; en GitHub Pages usa la ficha generada en el navegador. Todas las rutas son relativas para funcionar igual en «/» y en «/pwaauto/».

4.2 Estructura de carpetas

Ruta	Contenido
server.js · server/	Express: API, SSR, reseñas, push (render.js, push.js, almacen.js)
public/index.html · sw.js	App shell y service worker
public/manifest.json · offline.html	Manifiesto y página sin conexión
public/js/	22 módulos: enrutador, vistas, API, IndexedDB, reseñas, mapa, dispositivo, notificaciones…
public/data/	paradas.json (catálogo con fuentes) y trazos.json (recorridos precalculados)
public/img · fonts · icons · vendor	Fotos WebP y sprite SVG, tipografías, íconos de la app, Leaflet y supabase-js
scripts/	Generadores de íconos, claves VAPID, imágenes optimizadas y trazos
test/ · docs/	Pruebas automáticas; reporte de pruebas, SQL de Supabase y este documento
.github/workflows/pages.yml	Pruebas + despliegue automático en GitHub Pages

4.3 Renderizado en el cliente (CSR) y en el servidor (SSR)

CSR: el catálogo, «Cerca de mí», Avisos y Ajustes se dibujan en el navegador. Un enrutador basado en el hash (#/paradas, #/parada/id) cambia de vista sin recargar la página.

SSR: la ficha de cada parada (/parada/:id) llega del servidor como HTML completo, con salidas, horarios, fuentes y reseñas. Se ve aunque JavaScript esté desactivado (Figura 8). Después, ficha-ssr.js la «hidrata»: reutiliza ese HTML y le conecta favoritos, reseñas, mapa, recorridos y «Cómo llegar». La plantilla de la ficha (plantilla-ficha.js) es la misma para el servidor y el navegador.

4.4 Flujo de una reseña

1. El usuario escribe un comentario en la ficha de una parada.
2. La reseña se guarda primero en IndexedDB (almacén «pendientes») con un identificador único generado en el celular.
3. Se registra un Background Sync con la etiqueta «sync-resenas».
4. Cuando hay red, el service worker la envía a Supabase. Si falla, el navegador lo reintenta; gracias al identificador no se duplica.
5. Supabase la guarda aplicando las reglas de seguridad y el límite antispam.
6. Supabase Realtime avisa a los demás celulares con esa parada abierta y el comentario aparece sin recargar.

En navegadores sin Background Sync (Firefox, Safari) la página hace el envío al recuperar la conexión.

4.5 Estrategias de caché del service worker

Recurso	Estrategia	Motivo
App shell: HTML, CSS, JS, íconos, fuentes, Leaflet	Cache First (precache en install)	La app abre al instante y sin conexión
Datos: /api/*, paradas.json, trazos.json	Network First	Lo más reciente; si no hay red, la última copia
Fichas SSR /parada/:id	Network First → caché → ficha CSR	Una ficha nunca visitada se dibuja con IndexedDB
Imágenes	Stale-While-Revalidate	Se muestran rápido y se actualizan en segundo plano
Teselas de OpenStreetMap	Cache First con límite de 400	Zonas visitadas sin conexión, sin llenar el teléfono

4.6 Datos de Jalpan de Serra y sus fuentes

El catálogo contiene los dos puntos de salida de autobuses registrados en OpenStreetMap, con las corridas publicadas en sitios de venta y consulta de boletos (consultados el 10 de noviembre de 2026). Los datos que no se pudieron confirmar se muestran como «Consultar» (tarifa) o «Instalaciones por confirmar»; nada se inventó.

Destino	Empresa	Primera – Última	Salidas	Duración	Tarifa	Fuente
Xilitla	Grupo Vencedor	05:45 – 20:00	14 salidas al día	1 h 36 min	$140	Reservamos
Xilitla	Flecha Amarilla	04:30 – 19:15	10 salidas al día	1 h 45 min	$140	horariodeautobuses.com.mx
Río Verde	Grupo Vencedor	05:00 – 16:00	12 salidas al día	2 h 30 min	$260	Reservamos

Destino	Empresa	Primera – Última	Salidas	Duración	Tarifa	Fuente
Ciudad Valles	Grupo Vencedor	05:10 – 17:50	13 salidas al día	3 h 24 min	$486	Reservamos
Tampico	Grupo Vencedor	05:55 – 14:55	4 salidas al día	7 h	$1001	Reservamos
San Luis Potosí	Grupo Vencedor	05:00 – 16:00	05:00 · 07:00 · 13:00 · 16:00	—	Consultar	horariodeautobuses.com.mx
Querétaro	Flecha Amarilla / Primera Plus	05:30 – 20:30	Cada 2 horas	—	Consultar	terminal.horariodeautobuses.com.mx
Ciudad de México	Autobuses Coordinados	05:30 – 23:15	05:30 · 09:30 · 11:30 · 23:15	—	Consultar	horariodeautobuses.com.mx
Pinal de Amoles	Autobuses de la Sierra Gorda	06:45 – 22:00	Cada hora	—	Consultar	terminal.horariodeautobuses.com.mx

Punto de salida	Ubicación	Origen del dato
Central de Autobuses Jalpan de Serra	21.21589, -99.46300 · Policarpo Olvera s/n esq. Corregidora, Col. La Pedrera	Reservamos · Central de Autobuses Jalpan de Serra; horariodeautobuses.com.mx · Jalpan de Serra; horariodeautobuses.com.mx · Jalpan a Xilitla; terminal.horariodeautobuses.com.mx · Central de Jalpan; OpenStreetMap (ubicación)
Andén de Grupo Vencedor	21.21811, -99.47391 · Calle Heroico Colegio Militar	OpenStreetMap (andén de Grupo Vencedor); Reservamos · Grupo Vencedor desde Jalpan

Los recorridos por carretera se calcularon con OSRM sobre OpenStreetMap (scripts/generar-trazos.js); por ejemplo, Jalpan → Xilitla da 83 km, igual que la fuente de horarios. La ruta real del autobús puede variar y así se indica en la app.

4.7 Modelo de datos

Almacén	Contenido
paradas.json	id, nombre, tipo, direccion, zona, lat, lng, iluminada/techada/accesible (true, false o null), descripcion, consultado, fuentes[] y rutas[] (ruta, nombre, destino, empresa, primera, ultima, frecuencia, duracion, tarifa, fuente)
Supabase · resenas	id (uuid), parada_id, autor, texto (3–500), fecha, recibida, reportes, oculta
IndexedDB «rutaexpress»	paradas, resenas, pendientes (cola de envío), favoritos, meta (avisos recibidos, fecha de actualización)

Almacén	Contenido
localStorage	tema, nombre para comentarios, última parada consultada
Cache Storage	shell, datos, imágenes y mapas, separados por versión

5. Desarrollo por fases

Cada fase se desarrolló en orden, se verificó y se registró en un commit. Se incluyen los fragmentos clave del código tal como están en el repositorio.

Fase 0 · Entorno de desarrollo y repositorio

Se verificaron Node.js 24, npm 11 y Git 2.55, se inicializó el repositorio y se instalaron Express y web-push. El servidor mínimo sirve la carpeta public/ y responde /api/salud.

Servidor mínimo · server.js, líneas 12–23

```
const app = express();
const PUERTO = process.env.PORT || 3000;
const PUBLICO = path.join(__dirname, 'public');
const ARCHIVO_PARADAS = path.join(PUBLICO, 'data', 'paradas.json');

const leerParadas = () => JSON.parse(fs.readFileSync(ARCHIVO_PARADAS, 'utf8'));
const buscarParada = (id) => leerParadas().find((p) => p.id === id);

app.use(express.json());
app.use(express.static(PUBLICO));

app.get('/api/salud', (_req, res) => res.json({ ok: true, hora: new Date().toISOString() }));
```

Verificación: GET /api/salud → {"ok":true} (CP-01). **Commit:** `35e0061`

Fase 1 · App shell

El shell es la estructura fija de la app: cabecera con el indicador de conexión, área de contenido donde se dibujan las vistas y menú inferior. Se guarda en caché para abrir al instante.

Shell en index.html · public/index.html, líneas 42–61

```
  <header class="barra">
    <a class="marca" href="#/">
      <svg class="marca__logo" aria-hidden="true"><use href="img/iconos.svg#i-marca"></use></svg>
      <span class="marca__texto"><span class="marca__nombre">RutaExpress</span><span
class="marca__lugar">Jalpan de Serra</span></span>
    </a>
    <span class="conexion" data-conexion role="status" aria-live="polite" hidden></span>
  </header>

  <main id="contenido" class="contenido" tabindex="-1">
    <!-- Las vistas se dibujan aquí desde JavaScript (CSR) -->
    <div class="cargando">Cargando…</div>
  </main>

  <nav class="menu" aria-label="Navegación principal">
    <a href="#/" data-ruta="inicio"><svg class="ico" aria-hidden="true"><use href="img/iconos.svg#i-
inicio"></use></svg>Inicio</a>
    <a href="#/paradas" data-ruta="paradas"><svg class="ico" aria-hidden="true"><use
href="img/iconos.svg#i-paradas"></use></svg>Paradas</a>
    <a href="#/cerca" data-ruta="cerca"><svg class="ico" aria-hidden="true"><use
href="img/iconos.svg#i-cerca"></use></svg>Cerca</a>
    <a href="#/avisos" data-ruta="avisos"><svg class="ico" aria-hidden="true"><use
href="img/iconos.svg#i-avisos"></use></svg>Avisos</a>
    <a href="#/ajustes" data-ruta="ajustes"><svg class="ico" aria-hidden="true"><use
href="img/iconos.svg#i-ajustes"></use></svg>Ajustes</a>
  </nav>
```

Verificación: Cabecera, contenido y menú con 5 secciones (CP-02). **Commit:** `eaa0e32`

Fase 2 · Manifiesto e íconos

Un script propio genera los íconos PNG (normales, maskable, Apple y badge) sin dependencias. El manifiesto define nombre, colores, modo standalone, íconos y accesos directos.

manifest.json · public/manifest.json, líneas 3–15

```
  "name": "RutaExpress · Transporte en Jalpan de Serra",
  "short_name": "RutaExpress",
  "description": "Paradas de autobús y combis de Jalpan de Serra: rutas, horarios, tarifas y avisos de
desvío.",
  "lang": "es",
  "dir": "ltr",
  "start_url": "./?origen=pwa",
  "scope": "./",
  "display": "standalone",
  "display_override": ["window-controls-overlay", "standalone"],
  "orientation": "portrait",
  "background_color": "#f5eee3",
  "theme_color": "#b8452a",
  "categories": ["travel", "navigation", "utilities"],
```

Verificación: Manifiesto con 0 errores; 0 errores de instalabilidad (CP-03). **Commit:** `56eb2dd`

Fase 3 · Service worker básico

El registro usa rutas relativas a la carpeta de la app para funcionar en localhost, en /parada/x y en GitHub Pages. El SW precachea el shell en install y borra versiones viejas en activate.

Registro del service worker · public/js/registro-sw.js, líneas 58–68

```
export async function registrarServiceWorker() {
  if (!('serviceWorker' in navigator)) {
    console.warn('[SW] Este navegador no soporta service workers');
    return null;
  }
  try {
    // Rutas relativas a la raíz de la app: funcionan en localhost (/), en /parada/x y en GitHub Pages
(/repo/)
    const registro = await navigator.serviceWorker.register(url('sw.js'), { scope: url('./') });
    console.log('[SW] Registrado con alcance', registro.scope);
    vigilarActualizaciones(registro);
    return registro;
```

Eventos install y activate · public/sw.js, líneas 66–79

```
self.addEventListener('install', (evento) => {
  evento.waitUntil(caches.open(CACHE_SHELL).then((cache) => cache.addAll(SHELL)));
});

// ---------- activate: limpiar versiones viejas ----------
self.addEventListener('activate', (evento) => {
  evento.waitUntil(
    caches.keys().then((claves) =>
      Promise.all(
        claves.filter((c) => c.startsWith('rutaexpress-') && !CACHES_ACTUALES.includes(c)).map((c) =>
caches.delete(c))
      )
    )
```

```
  );
});
```

Verificación: Estado: activated · alcance http://localhost:3100/ (CP-04). **Commit:** `0f8daf5`

Fase 4 · Renderizado CSR y SSR

El servidor genera la ficha completa con las reseñas incluidas; en el navegador, un enrutador por hash dibuja las vistas sin recargar.

Ruta SSR de la ficha · server.js, líneas 75–87

```
app.get('/parada/:id', async (req, res) => {
  const parada = buscarParada(req.params.id);
  if (!parada) return res.status(404).type('html').send(paginaNoEncontrada());
  let resenas = resenasDe(parada.id);
  if (usarSupabase()) {
    try {
      resenas = await Remoto.listarResenas(parada.id, { señal: AbortSignal.timeout(2500) });
    } catch (e) {
      console.warn('[SSR] Supabase no respondió, se usan las reseñas locales:', e.message);
    }
  }
  res.type('html').send(paginaFicha(parada, resenas));
});
```

Enrutador CSR · public/js/router.js, líneas 6–12

```
export function ruta(patron, vista, nombreMenu) {
  const claves = [];
  const regex = new RegExp(
    '^' + patron.replace(/:(\w+)/g, (_, k) => (claves.push(k), '([^/]+)')) + '$'
  );
  rutas.push({ regex, claves, vista, nombreMenu });
}
```

Verificación: Sin JavaScript se ve «Central de Autobuses Jalpan de Serra · 9 corridas» (CP-06); 2 paradas; buscar «querétaro» deja 1 sin recargar (#/paradas) (CP-05). **Commit:** `c350c81`

Fase 5 · Splash y Home

El splash del sistema sale del manifiesto; dentro de la app hay un splash animado. El Home tiene el botón «Instalar app», que guarda el evento beforeinstallprompt y, si el navegador no lo entrega, muestra instrucciones.

Captura del evento de instalación · public/js/instalacion.js, líneas 76–94

```
export function iniciarInstalacion() {
  const guardar = (e) => {
    e.preventDefault(); // lo guardamos para lanzarlo desde nuestro botón
    eventoInstalacion = e;
    avisar();
  };
  window.addEventListener('beforeinstallprompt', guardar);
  // Por si llegó antes de que cargara este módulo
  window.addEventListener('instalacion-disponible', () => {
    if (window.__eventoInstalacion) {
      eventoInstalacion = window.__eventoInstalacion;
      avisar();
    }
```

```
  });
  window.addEventListener('appinstalled', () => {
    eventoInstalacion = null;
    avisar();
  });
}
```

Verificación: Splash visible al cargar: true; oculto tras cargar la vista: true (CP-08). **Commit:** `60ef834`

Fase 6 · Datos locales

IndexedDB guarda el respaldo del catálogo, las favoritas, la copia de reseñas y la cola de envíos; el mismo archivo db.js lo usan la página y el service worker. Las preferencias pequeñas van en localStorage.

Apertura de IndexedDB y creación de almacenes · public/js/db.js, líneas 9–30

```
  function abrir() {
    if (promesaDB) return promesaDB;
    promesaDB = new Promise((resolver, rechazar) => {
      const pet = indexedDB.open(NOMBRE, VERSION);
      pet.onupgradeneeded = () => {
        const db = pet.result;
        if (!db.objectStoreNames.contains('paradas')) db.createObjectStore('paradas', { keyPath:
'id' });
        if (!db.objectStoreNames.contains('resenas')) {
          db.createObjectStore('resenas', { keyPath: 'id' }).createIndex('paradaId', 'paradaId');
        }
        if (!db.objectStoreNames.contains('pendientes')) db.createObjectStore('pendientes', { keyPath:
'id' });
        if (!db.objectStoreNames.contains('favoritos')) db.createObjectStore('favoritos', { keyPath:
'id' });
        if (!db.objectStoreNames.contains('meta')) db.createObjectStore('meta', { keyPath: 'clave' });
      };
      pet.onsuccess = () => resolver(pet.result);
      pet.onerror = () => {
        promesaDB = null;
        rechazar(pet.error);
      };
    });
    return promesaDB;
  }
```

Verificación: Almacén «paradas» de IndexedDB con 2 registros (CP-10); Tras recargar: data-tema y localStorage = oscuro · oscuro (CP-11). **Commit:** `4b5e2bc`

Fase 7 · Estrategias de caché y offline

El service worker decide la estrategia según el recurso (tabla 4.5). Si no hay red, Network First responde con la última copia guardada.

Estrategia Network First · public/sw.js, líneas 90–102

```
async function networkFirst(peticion, { nombreCache = CACHE_DATOS, respaldo } = {}) {
  const cache = await caches.open(nombreCache);
  try {
    const respuesta = await fetch(peticion);
    if (respuesta.ok) cache.put(peticion, respuesta.clone());
    return respuesta;
  } catch (error) {
```

```
    const enCache = await caches.match(peticion); // busca en todas las cachés (incluye el precache)
    if (enCache) return enCache;
    if (respaldo) return respaldo();
    throw error;
  }
}
```

Elección de estrategia en el evento fetch · public/sw.js, líneas 170–184

```
  // 2) Datos → red primero
  if (relativa.startsWith('api/') || relativa.startsWith('data/')) {
    if (relativa.startsWith('api/salud') || relativa.startsWith('api/push')) return; // siempre a la
red
    evento.respondWith(networkFirst(request));
    return;
  }

  // 3) Imágenes → stale-while-revalidate
  if (request.destination === 'image') {
    evento.respondWith(staleWhileRevalidate(request));
    return;
  }

  // 4) Shell estático (CSS, JS, manifiesto) → caché primero
  evento.respondWith(cacheFirst(request));
```

Verificación: Servidor apagado: 2 paradas · indicador «Sin conexión» (CP-13); Ficha nunca visitada → /#/parada/anden-vencedor · Andén de Grupo Vencedor (CP-15). **Commit:** `4518bc0`

Fase 8 · Reseñas y Background Sync

Guardar local primero, enviar después: la reseña entra a la cola de IndexedDB y se registra un sync; el service worker la envía cuando hay red.

Guardar en la cola y registrar el sync · public/js/resenas.js, líneas 59–75

```
export async function publicarResena({ paradaId, autor, texto }) {
  const resena = { id: uuid(), paradaId, autor: autor || 'Anónimo', texto, fecha: new
Date().toISOString() };
  await DB.agregarPendiente(resena);

  const registro = await navigator.serviceWorker?.ready.catch(() => null);
  if (registro && 'sync' in registro) {
    try {
      await registro.sync.register(ETIQUETA_SYNC);
      return { resena, modo: 'background-sync' };
    } catch (e) {
      console.warn('[Sync] No se pudo registrar, se envía desde la página', e);
    }
  }
  // Respaldo sin Background Sync (Firefox, Safari): intento inmediato
  const enviadas = await enviarPendientes();
  return { resena, modo: enviadas.includes(resena.id) ? 'enviada' : 'en-cola' };
}
```

Envío de la cola desde el service worker · public/sw.js, líneas 190–210

```
async function enviarResenasPendientes() {
  const pendientes = await self.RutaDB.leerPendientes();
  const enviadas = [];
  for (const resena of pendientes) {
    // Si no hay red, fetch lanza error → el navegador reintentará el sync más tarde
```

```
    let resultado;
    if (self.RutaRemoto.activo()) {
      resultado = await self.RutaRemoto.publicarResena(resena); // Supabase
    } else {
      const r = await fetch(URL_RESENAS, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(resena),
      }); // servidor Node
      resultado = { ok: r.ok, reintentar: !r.ok && r.status !== 400, estado: r.status };
    }
    if (resultado.ok) {
      await self.RutaDB.guardarResena(resena);
      await self.RutaDB.borrarPendiente(resena.id);
      enviadas.push(resena.id);
    } else if (!resultado.reintentar) {
```

Evento sync · public/sw.js, líneas 230–234

```
self.addEventListener('sync', (evento) => {
  if (evento.tag === 'sync-resenas') {
    evento.waitUntil(enviarResenasPendientes());
  }
});
```

Verificación: Sin conexión: 1 pendiente(s) · cola IDB: 1. Al reconectar: cola 0, reseña en el servidor: true (CP-17). **Commit:** `b57e283`

Fase 9 · Ciclo de vida del service worker

La versión nueva queda en espera hasta que el usuario pulsa «Actualizar»; entonces la página envía SKIP_WAITING y el SW nuevo toma el control con clients.claim().

skipWaiting y clients.claim · public/sw.js, líneas 239–247

```
self.addEventListener('message', (evento) => {
  if (evento.data?.tipo === 'SKIP_WAITING') self.skipWaiting();
  if (evento.data?.tipo === 'VERSION') evento.source?.postMessage({ tipo: 'VERSION', version:
VERSION });
});

// Al activarse, toma el control de las pestañas abiertas sin esperar a recargarlas
self.addEventListener('activate', (evento) => {
  evento.waitUntil(self.clients.claim());
});
```

Detección de la versión nueva en la página · public/js/registro-sw.js, líneas 35–41

```
  registro.addEventListener('updatefound', () => {
    const nuevo = registro.installing;
    nuevo?.addEventListener('statechange', () => {
      // "installed" + ya hay un SW controlando = es una ACTUALIZACIÓN (no la primera instalación)
      if (nuevo.state === 'installed' && navigator.serviceWorker.controller)
mostrarBannerActualizacion(registro);
    });
  });
```

Verificación: Versión v13 → banner visible: true → tras «Actualizar»: v99-prueba (CP-19).
Commit: `5c7c538`

Fase 10 · Notificaciones locales y push

El servidor guarda las suscripciones y envía los avisos firmados con VAPID; elimina las suscripciones caducadas. El service worker muestra la notificación y la guarda en el historial.

Envío a todos los suscriptores (servidor) · server/push.js, líneas 34–52

```
async function enviarATodos(aviso) {
  const suscripciones = leerSuscripciones();
  const carga = JSON.stringify(aviso);
  const caducadas = new Set();
  let enviadas = 0;
  await Promise.all(
    suscripciones.map(async (s) => {
      try {
        await webpush.sendNotification(s, carga, { TTL: 60 * 60 * 6 });
        enviadas++;
      } catch (e) {
        if (e.statusCode === 404 || e.statusCode === 410) caducadas.add(s.endpoint);
        else console.warn('[push] Error al enviar', e.statusCode, e.body || e.message);
      }
    })
  );
  if (caducadas.size) guardarSuscripciones(leerSuscripciones().filter((s)
=> !caducadas.has(s.endpoint)));
  return { total: suscripciones.length, enviadas, eliminadas: caducadas.size };
}
```

Evento push en el service worker · public/sw.js, líneas 258–287

```
self.addEventListener('push', (evento) => {
  let aviso;
  try {
    aviso = evento.data ? evento.data.json() : null;
  } catch {
    aviso = { titulo: 'RutaExpress', cuerpo: evento.data?.text() };
  }
  aviso = { id: `${Date.now()}`, titulo: 'RutaExpress', cuerpo: 'Hay un aviso nuevo.', url:
'./#/avisos', tipo: 'info', fecha: new Date().toISOString(), ...aviso };

  evento.waitUntil(
    (async () => {
      await guardarAviso(aviso);
      const clientes = await self.clients.matchAll({ type: 'window', includeUncontrolled: true });
      clientes.forEach((c) => c.postMessage({ tipo: 'AVISO_RECIBIDO', aviso }));
      await self.registration.showNotification(`${ICONOS_AVISO[aviso.tipo] || ''}
${aviso.titulo}`.trim(), {
        body: aviso.cuerpo,
        icon: './icons/icon-192.png',
        badge: './icons/badge-72.png',
        tag: `aviso-${aviso.tipo}`,
        renotify: true,
        vibrate: [200, 100, 200],
        data: { url: new URL(aviso.url, self.location).href },
        actions: [
          { action: 'ver', title: 'Ver detalles' },
          { action: 'cerrar', title: 'Cerrar' },
        ],
      });
    })()
  );
});
```

Verificación: Suscripción en wns2-bn3p.notify.windows.com; envío 1/1; aviso recibido por el SW: «Retraso en corridas a Xilitla» (CP-21). **Commit:** `0ebea3f`

Fase 11 · Dispositivo e hidratación

El módulo de dispositivo calcula distancias con Haversine, el rumbo y el tiempo caminando; la ficha SSR se hidrata con los datos incrustados por el servidor.

Distancia con la fórmula de Haversine · public/js/dispositivo.js, líneas 24–31

```
export function distanciaMetros(a, b) {
  const R = 6371000;
  const rad = (g) => (g * Math.PI) / 180;
  const dLat = rad(b.lat - a.lat);
  const dLng = rad(b.lng - a.lng);
  const h = Math.sin(dLat / 2) ** 2 + Math.cos(rad(a.lat)) * Math.cos(rad(b.lat)) * Math.sin(dLng / 2)
** 2;
  return 2 * R * Math.asin(Math.sqrt(h));
}
```

Hidratación de la ficha SSR · public/js/ficha-ssr.js, líneas 15–29

```
const datos = JSON.parse(document.getElementById('datos-ssr')?.textContent || 'null');
const raiz = document.querySelector('.ficha');

if (datos?.parada && raiz) {
  // Guardamos la parada en el respaldo local por si luego se abre sin conexión
  window.RutaDB.leerParadas()
    .then((lista) => {
      if (!lista.some((p) => p.id === datos.parada.id)) return window.RutaDB.guardarParadas([...lista,
datos.parada]);
    })
    .catch(() => {});
  activarAccionesFicha(raiz, datos.parada).then(() => {
    raiz.dataset.hidratada = 'true';
    console.log('[SSR] Ficha hidratada:', datos.parada.id);
  });
}
```

Verificación: Cerca de mí: Andén de Grupo Vencedor · 424 m al oeste · unos 5 min (CP-22); Hidratada: true · ★ En favoritas / Cómo llegar / Compartir (CP-23). **Commit:** `68cdada`

Fase 12 · Pruebas

Diez pruebas automáticas del servidor con node:test, verificación de extremo a extremo en Edge y auditoría Lighthouse (sección 7).

Prueba de reintento sin duplicados · test/servidor.test.js, líneas 60–66

```
test('CP-06 · reenviar la misma reseña (reintento de Background Sync) no la duplica', async () => {
  const resena = { id: 'test-2', paradaId: 'anden-vencedor', texto: 'Hay sala de espera techada',
fecha: new Date().toISOString() };
  assert.equal((await post('/api/resenas', resena)).status, 201);
  assert.equal((await post('/api/resenas', resena)).status, 200);
  const lista = await (await fetch(`${base}/api/paradas/anden-vencedor/resenas`)).json();
  assert.equal(lista.filter((r) => r.id === 'test-2').length, 1);
});
```

Verificación: 10 de 10 pruebas automáticas pasan. **Commit:** `8d8abaf`

Fase 13 · Publicación

El workflow ejecuta las pruebas y publica public/ en GitHub Pages en cada push a main.

Workflow de GitHub Actions · .github/workflows/pages.yml, líneas 18–44

```
jobs:
  pruebas:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm test

  publicar:
    needs: pruebas
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.despliegue.outputs.page_url }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with:
          path: public
      - id: despliegue
        uses: actions/deploy-pages@v4
```

Verificación: Sitio publicado y auditado con Lighthouse (CP-25, CP-26). **Commit:** `61afe88`

6. Mejoras posteriores

6.1 Mapa real, recorridos y «Cómo llegar»

Leaflet con mapas de OpenStreetMap en el catálogo, «Cerca de mí» y la ficha. En la ficha, tocar una corrida dibuja su recorrido por carretera hasta el destino (trazos precalculados con OSRM y guardados en la app para verse sin conexión), y «Cómo llegar» dibuja el camino a pie desde la ubicación del usuario sin salir de la app.

Dibujo del recorrido de una corrida · public/js/mapa.js, líneas 137–144

```
    mostrarCorrida(trazo, destino, color) {
      capa.clearLayers();
      const l = linea(trazo.puntos, color);
      L.circleMarker([destino.lat, destino.lng], { radius: 8, color: '#fff', weight: 3, fillColor:
color, fillOpacity: 1 })
        .bindTooltip(destino.nombre, { permanent: true, direction: 'top', offset: [0, -8] })
        .addTo(capa);
      mapa.fitBounds(l.getBounds().pad(0.12));
    },
```

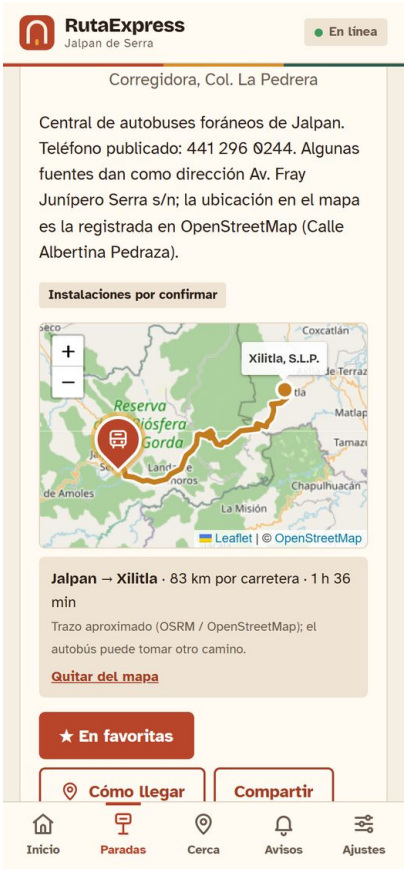


Figura 5. Recorrido de la corrida a Xilitla en el mapa

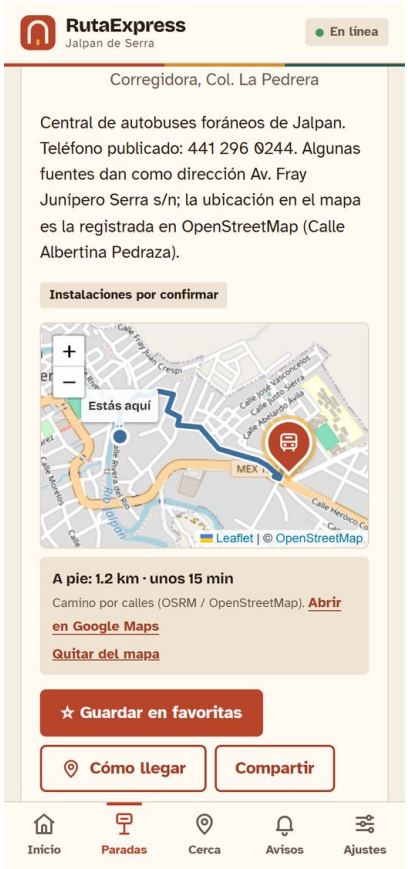


Figura 6. «Cómo llegar»: camino a pie dentro de la app

6.2 Datos reales de Jalpan

Se reemplazaron las paradas de ejemplo por la central de autobuses y el andén de Grupo Vencedor, con ubicaciones de OpenStreetMap y corridas de fuentes públicas citadas en cada ficha (Figuras 3 y 4).

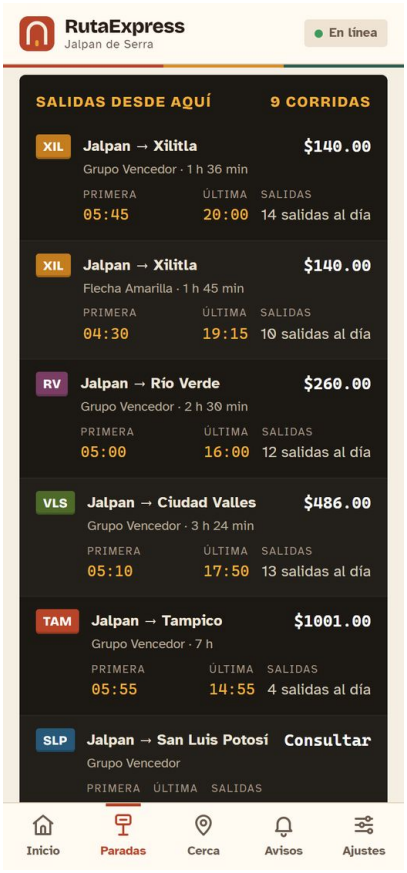


Figura 3. Tablero de salidas de la central

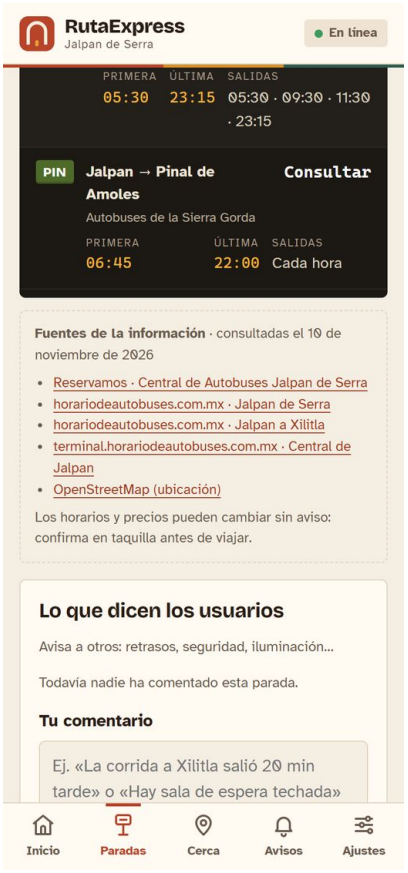


Figura 4. Fuentes de la información en la ficha

6.3 Instalación más robusta

El botón «Instalar app» ya no se queda sin respuesta: si el navegador no entrega el diálogo nativo, muestra instrucciones para Chrome, Edge, iPhone, Firefox, Samsung Internet o navegadores dentro de otras apps.

6.4 Diseño con identidad local

Paleta tomada de la Sierra Gorda (terracota y ocre de las misiones, verde de la sierra, cantera), tipografías propias, íconos SVG, letreros de ruta con color fijo y un tablero de horarios inspirado en los andenes de una terminal.

6.5 Reseñas reales con Supabase

Las reseñas se comparten entre todos los usuarios también en GitHub Pages y llegan en tiempo real. Reglas de seguridad: cualquiera puede leer y publicar, nadie puede editar ni borrar desde la app, máximo 8 comentarios por minuto por parada y ocultamiento automático con 3 reportes.

Reglas Row Level Security · docs/supabase.sql, líneas 21–36

```
alter table public.resenas enable row level security;

drop policy if exists "Leer reseñas visibles" on public.resenas;
create policy "Leer reseñas visibles" on public.resenas
  for select to anon, authenticated
  using (not oculta);

drop policy if exists "Publicar reseñas" on public.resenas;
create policy "Publicar reseñas" on public.resenas
  for insert to anon, authenticated
  with check (
    not oculta and reportes = 0
    and fecha <= now() + interval '5 minutes'
    and fecha >= now() - interval '30 days'          -- una reseña offline puede llegar días después
  );
-- (No hay políticas de UPDATE ni DELETE: nadie puede modificar ni borrar desde la app.)
```

7. Pruebas

7.1 Herramientas

Herramienta	Uso
node:test (npm test)	Pruebas automáticas del servidor: API, SSR, reseñas, push y archivos PWA
Edge + Chrome DevTools Protocol	Pruebas de extremo a extremo: servidor apagado de verdad, GPS simulado, push real, actualización del SW y capturas
Lighthouse 13.5 (perfil móvil)	Auditoría de rendimiento, accesibilidad, buenas prácticas y SEO de la app publicada
DevTools › Application	Revisar manifiesto, service worker, Cache Storage, IndexedDB y localStorage
GitHub Actions	Ejecutar las pruebas antes de cada publicación

7.2 Pruebas automáticas del servidor

Caso	Qué se comprueba	Resultado
T-01	/api/salud responde correctamente	Pasa
T-02	El catálogo trae paradas con rutas y horarios (tarifa numérica o «por consultar»)	Pasa
T-03	La ficha SSR llega completa con fuentes y datos para hidratar	Pasa
T-04	Una parada inexistente devuelve 404	Pasa
T-05	Una reseña publicada aparece en la API y en la ficha SSR	Pasa
T-06	Reenviar la misma reseña no la duplica	Pasa
T-07	Se rechazan reseñas inválidas y el HTML se escapa (prevención de XSS)	Pasa
T-08	Clave pública VAPID y validación de suscripciones	Pasa
T-09	Manifiesto, service worker con sus 6 eventos y offline.html	Pasa
T-10	Todos los archivos precacheados por el SW existen	Pasa

Resultado: 10 de 10 pruebas pasan (npm test).

7.3 Casos de prueba completos

Resultado: Pasa · Parcial · Pendiente. Las pruebas se hicieron en Edge controlado por script (viewport de celular 390 × 844) contra el servidor local, salvo CP-25 y CP-26, que se hicieron contra la app publicada. En los casos de reseñas se usó la API del servidor local para no llenar la base real de Supabase con comentarios de prueba; Supabase se verificó aparte (CP-29).

ID	Fase	Caso	Resultado obtenido	Estado	Evid.
CP-01	0	Servidor arranca	GET /api/salud → {"ok":true}	Pasa	—
CP-02	1	App shell	Cabecera, contenido y menú con 5 secciones	Pasa	Fig. 1

ID	Fase	Caso	Resultado obtenido	Estado	Evid.
CP-03	2	Manifiesto e instalabilidad	Manifiesto con 0 errores; 0 errores de instalabilidad	Pasa	—
CP-04	3	Registro del service worker	Estado: activated · alcance http://localhost:3100/	Pasa	—
CP-05	4	Catálogo CSR y buscador	2 paradas; buscar «querétaro» deja 1 sin recargar (#/paradas)	Pasa	Fig. 2
CP-06	4	Ficha SSR sin JavaScript	Sin JavaScript se ve «Central de Autobuses Jalpan de Serra · 9 corridas»	Pasa	Fig. 8
CP-07	5	Splash del sistema (app instalada)	Requiere instalar en un teléfono Android y abrir desde el ícono	Pendiente	—
CP-08	5	Splash dentro de la app	Splash visible al cargar: true; oculto tras cargar la vista: true	Pasa	Fig. 15
CP-09	5	Botón «Instalar app»	Evento beforeinstallprompt capturado: true; botón «Instalar app» visible: true. El diálogo nativo requiere un clic real del usuario	Parcial	Fig. 1
CP-10	6	IndexedDB	Almacén «paradas» de IndexedDB con 2 registros	Pasa	—
CP-11	6	localStorage (tema)	Tras recargar: data-tema y localStorage = oscuro · oscuro	Pasa	Fig. 14
CP-12	6	Favoritas	«Mis paradas» muestra la central como «Favorita»	Pasa	—
CP-13	7	Offline: shell y catálogo	Servidor apagado: 2 paradas · indicador «Sin conexión»	Pasa	Fig. 10
CP-14	7	Offline: ficha visitada	Ficha visitada servida desde la caché: Central de Autobuses Jalpan de Serra · RutaExpress · ssr=ficha	Pasa	—
CP-15	7	Offline: ficha no visitada	Ficha nunca visitada → /#/parada/anden-vencedor · Andén de Grupo Vencedor	Pasa	Fig. 16
CP-16	8	Reseña con conexión	El servidor devuelve 1 reseña(s) de la central	Pasa	—
CP-17	8	Reseña offline + Background Sync	Sin conexión: 1 pendiente(s) · cola IDB: 1. Al reconectar: cola 0, reseña en el servidor: true	Pasa	Fig. 9
CP-18	8	Sin duplicados al reintentar	El servidor guarda una sola copia al recibir dos veces el mismo id (prueba automática T-06)	Pasa	—
CP-19	9	Nueva versión del SW	Versión v13 → banner visible: true → tras «Actualizar»: v99-prueba	Pasa	Fig. 11
CP-20	10	Notificación local	registration.getNotifications() devuelve 1 notificación(es)	Pasa	—
CP-21	10	Notificación push	Suscripción en wns2-bn3p.notify.windows.com; envío 1/1; aviso recibido por el SW: «Retraso en corridas a Xilitla»	Pasa	Fig. 12
CP-22	11	Cerca de mí (GPS)	Ubicación simulada (21.2170, -99.4700): Andén de Grupo Vencedor · 424 m al oeste · unos 5 min caminando	Pasa	Fig. 7

ID	Fase	Caso	Resultado obtenido	Estado	Evid.
CP-23	11	Hidratación de la ficha SSR	Hidratada: true · ★ En favoritas / Cómo llegar / Compartir	Pasa	—
CP-24	11	Vibración y compartir	Requieren un teléfono Android (en escritorio no hay vibración)	Pendiente	—
CP-25	12	Lighthouse (móvil)	Rendimiento 99, accesibilidad 96, buenas prácticas 96, SEO 100	Pasa	Tabla 7.4
CP-26	13	GitHub Pages	URL publicada responde 200; manifiesto e instalabilidad sin errores; SW con alcance /pwaauto/	Pasa	—
CP-27	Mejora	Recorrido de una corrida	Al tocar la corrida: «Jalpan → Xilitla · 83 km por carretera · 1 h 36 min»	Pasa	Fig. 5
CP-28	Mejora	«Cómo llegar» a pie	Desde (21.2170, -99.4700) a la central: A pie 1.2 km · unos 15 min, camino por calles, sin salir de la app	Pasa	Fig. 6
CP-29	Mejora	Reseñas en Supabase	La tabla responde con las reglas RLS; la reseña publicada desde la app se guarda y llega en tiempo real a otro dispositivo (verificado por el equipo)	Pasa	—

Pendientes: CP-07 y CP-24 necesitan un teléfono Android. CP-09 es parcial porque el diálogo nativo de instalación solo se abre con un clic real de una persona; se comprobó que el navegador entrega el evento y que el botón aparece.

7.4 Auditoría Lighthouse

Auditoría de la app publicada (https://kaidav54.github.io/pwaauto/) con Lighthouse 13.5, perfil móvil, en Microsoft Edge.

Categoría	Puntuación
Rendimiento	99
Accesibilidad	96
Buenas prácticas	96
SEO	100

Métrica	Valor
First Contentful Paint	1,1 s
Largest Contentful Paint	2,0 s
Total Blocking Time	30 ms
Cumulative Layout Shift	0
Speed Index	2,8 s

La primera auditoría encontró dos detalles que se corrigieron antes de esta medición: el contraste de los títulos pequeños en el tema oscuro (3.2:1, ahora con el color de enlace) y un error 404 en la consola porque la app preguntaba por /api/salud en GitHub Pages (ahora no pregunta en github.io).

8. Publicación

8.1 Repositorio y GitHub Pages

1. Se subió el repositorio a github.com/Kaidav54/pwaauto.
2. En Settings › Pages se eligió como origen «GitHub Actions».
3. El workflow pages.yml ejecuta las pruebas y publica la carpeta public/ en cada push a main.
4. La aplicación queda en kaidav54.github.io/pwaauto.

8.2 Qué funciona en GitHub Pages y qué necesita el servidor

Función	GitHub Pages	Servidor Node
Instalación, splash, offline y caché	Sí	Sí
Catálogo, mapa, recorridos y «Cómo llegar»	Sí	Sí
Ficha de parada	Generada en el navegador	Generada en el servidor + hidratación
Reseñas, Background Sync y tiempo real	Sí (Supabase)	Sí (Supabase)
Cerca de mí, vibrar, compartir, notificación local	Sí	Sí
Notificaciones push	No: enviarlas requiere un servidor	Sí

8.3 Cómo publicar una actualización

1. Hacer los cambios y subir VERSION en public/sw.js.
2. Ejecutar npm test.
3. Hacer commit y push a main; GitHub Actions publica la nueva versión.
4. Los usuarios ven «Hay una nueva versión» y al pulsar «Actualizar» se activa.

9. Capturas de pantalla

Capturas en un viewport de celular (390 × 844). Las figuras 5, 6, 3 y 4 están en la sección 6.

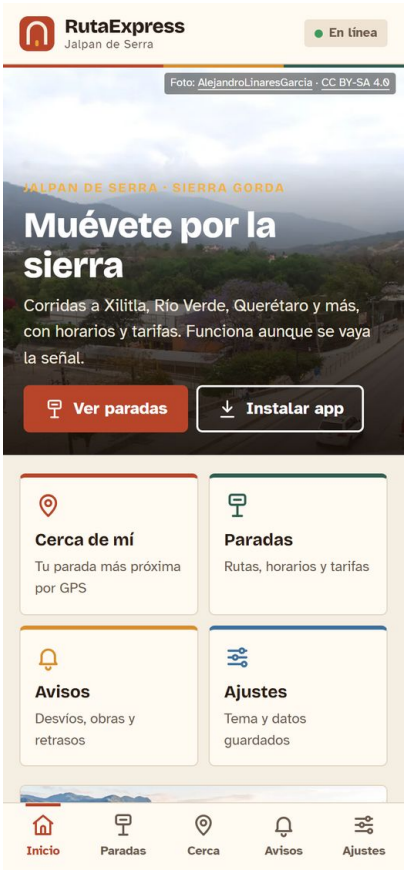


Figura 1. Inicio con la foto de Jalpan y el botón de instalación



Figura 2. Catálogo con buscador y mapa de Jalpan



Figura 8. Ficha SSR con JavaScript desactivado

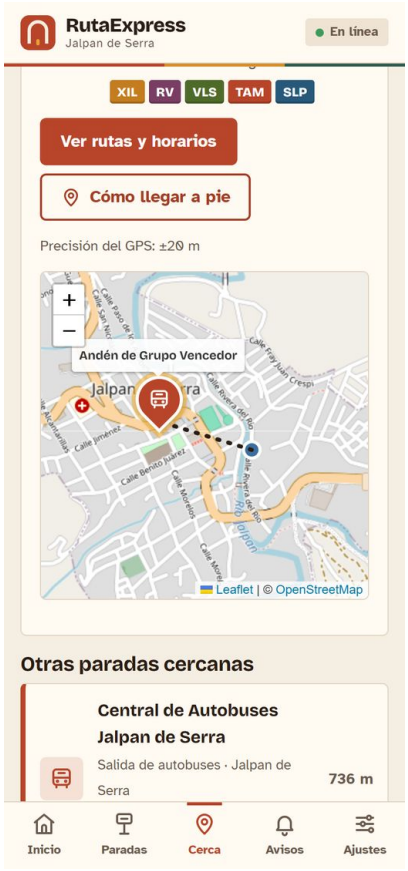


Figura 7. «Cerca de mí» con ubicación simulada en Jalpan



Figura 9. Reseña escrita sin conexión, pendiente de enviar



Figura 10. Catálogo con el servidor apagado



Figura 16. Ficha nunca visitada, abierta sin conexión



Figura 11. Aviso de nueva versión del service worker



Figura 12. Aviso push recibido y panel de avisos

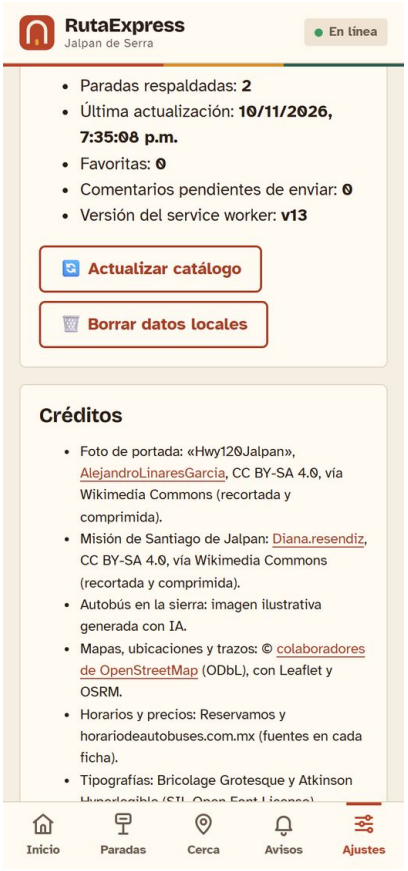


Figura 13. Créditos de fotos, mapas, datos y tipografías



Figura 15. Splash dentro de la app al abrir



Figura 14. Tema oscuro

10. Créditos de imágenes y datos

Recurso	Autor / fuente	Licencia
Foto de portada de la app: carretera federal 120 en Jalpan	AlejandroLinaresGarcia, Wikimedia Commons («Hwy120Jalpan.JPG»)	CC BY-SA 4.0 (recortada y comprimida)
Foto de la Misión de Santiago de Jalpan	Diana.resendiz, Wikimedia Commons	CC BY-SA 4.0 (recortada y comprimida)
Autobús en la sierra	Imagen ilustrativa generada con IA	Se indica en la app
Mapas, ubicaciones y direcciones	© colaboradores de OpenStreetMap	ODbL
Recorridos por carretera y a pie	OSRM (routing.openstreetmap.de) sobre OpenStreetMap	ODbL
Horarios y tarifas	Reservamos; horariodeautobuses.com.mx (consultados el 10/11/2026)	Citados en cada ficha
Tipografías	Bricolage Grotesque, Atkinson Hyperlegible (Google Fonts)	SIL Open Font License
Leaflet / supabase-js	Proyectos de código abierto	BSD-2 / MIT

La sección «Créditos» de la app (Ajustes) muestra la misma información (Figura 13).

11. Limitaciones y trabajo futuro

- **Datos por confirmar en taquilla.** Los horarios y tarifas salen de sitios públicos que no muestran fecha de actualización y a veces no coinciden. Faltan las tarifas de San Luis Potosí, Querétaro, Ciudad de México y Pinal de Amoles, y las instalaciones de cada punto (iluminación, techo, accesibilidad).
- **Transporte local.** Las bases de combis hacia las comunidades no se incluyeron porque no hay información pública confiable; se pueden agregar con un levantamiento en campo.
- **Push en la versión publicada.** GitHub Pages no puede enviar notificaciones; para tenerlas en línea hay que desplegar el servidor Node (por ejemplo en Render o Railway).
- **Pruebas en dispositivo.** Quedan pendientes el splash del sistema, la vibración y la hoja de compartir en un teléfono Android (CP-07, CP-24).
- **iPhone.** La instalación es manual (Compartir › Agregar a inicio), no hay Background Sync y el push solo funciona con la app instalada.

12. Conclusiones

Se construyó una Aplicación Web Progresiva completa siguiendo las 13 fases de la guía, con un commit por fase y verificación en cada paso. La aplicación cumple los cuatro resultados esperados: se instala, funciona sin conexión, sincroniza las reseñas al recuperar la red y recibe notificaciones push; además obtuvo puntuaciones de 96 a 100 en Lighthouse.

El service worker resultó la pieza central: además de la caché, coordina el envío en segundo plano, las notificaciones y las actualizaciones. Combinar CSR y SSR con una plantilla compartida permitió tener fichas que se ven sin JavaScript y que, una vez hidratadas, ofrecen mapa, recorridos y comentarios en tiempo real.

Con los datos reales de Jalpan de Serra, citados con sus fuentes, la aplicación ya es útil para quien viaja desde la Sierra Gorda; el siguiente paso es confirmar en campo las tarifas y las instalaciones de cada punto de salida.

Anexo A. Enlaces y comandos

Recurso	Dirección
Repositorio	https://github.com/Kaidav54/pwaauto
Aplicación publicada	https://kaidav54.github.io/pwaauto/
Script de base de datos	docs/supabase.sql
Reporte de pruebas	docs/PRUEBAS.md

Comando	Qué hace
npm install / npm start	Instala dependencias / inicia el servidor en http://localhost:3000
npm test	Ejecuta las pruebas automáticas
npm run iconos	Regenera los íconos de la app
npm run imagenes	Optimiza las fotos a WebP
npm run trazos	Recalcula los recorridos por carretera
npm run vapid	Genera las claves VAPID para push

Anexo B. Historial de commits

Commit	Descripción
35e0061|Fase 0: entorno, dependencias y servidor mínimo	
eaa0e32|Fase 1: app shell (index.html) y hoja de estilos	
56eb2dd|Fase 2: manifiesto e íconos generados	
0f8daf5|Fase 3: registro del service worker y sw.js con install/activate/fetch	
c350c81|Fase 4: catálogo remoto, enrutador CSR y ficha SSR	
60ef834|Fase 5: splash dentro de la app y Home con botón de instalación	

Commit	Descripción
4b5e2bc|Fase 6: IndexedDB, preferencias en localStorage y respaldo local del catálogo	
4518bc0|Fase 7: offline.html, estrategias de caché e indicador de conexión	
b57e283|Fase 8: API de reseñas, cola local y Background Sync	
5c7c538|Fase 9: ciclo de vida del SW, skipWaiting/clients.claim y aviso de nueva versión	
0ebea3f|Fase 10: notificaciones locales y push con VAPID	
68cdada|Fase 11: módulo de dispositivo, vista Cerca de mí e hidratación de la ficha SSR	
8d8abaf|Fase 12: pruebas automáticas del servidor y plantilla del reporte de pruebas	
61afe88|Fase 13: workflow de GitHub Pages y guía de publicación	
3509d6f|Docs: enlaces del repositorio y de GitHub Pages	
9be321b|Mapa real con Leaflet/OpenStreetMap e instalación con instrucciones de respaldo	
3ddf365|Rediseño con identidad de Jalpan de Serra	
42161cf|Reseñas reales compartidas con Supabase	

Commit	Descripción
e26c63c|Docs: documentación del proyecto en Word	
c46a30d|Docs: documentación final con portada (Word y PDF)	
b7e0b76|Datos reales de Jalpan de Serra: central y andén de Vencedor con sus corridas	
66a4565|Mapa de la ficha: recorrido de cada corrida y camino a pie dentro de la app	
dc393c3|Fotos con licencia libre de Wikimedia Commons y créditos	
a4544e0|Lighthouse: contraste de los títulos pequeños y sin consulta a /api/salud en GitHub Pages; SW v13	